Nested-Logit Hierarchical Multi-Agent Reinforcement Learning for Real-Time Task Allocation in Robotic Warehouses

Xuchen He Vijay K. Madisetti

Abstract

We propose a Nested-Logit-based Hierarchical Multi-Agent Reinforcement Learning (NL-HMARL) framework for real-time task allocation in robotic warehouses. The manager first selects a task nest, then chooses a specific task within the selected nest, thereby capturing intra-nest task correlations while remaining end-to-end trainable. We formalize the problem definition, present the policy and training objectives, provide algorithmic descriptions, and prove that inference complexity is linear in the number of tasks. Through systematic experiments across 6 configurations (3 difficulty levels $\times$ 2 environment scales), we demonstrate that NL-HMARL outperforms standard Softmax-HMARL in complex coordination scenarios with high task density and low value differentiation, exhibiting more pronounced advantages in large-scale environments.

Keywords: Warehouse automation; Multi-agent reinforcement learning; Nested Logit; Hierarchical reinforcement learning; Discrete event simulation

1 Introduction

1.1 Background

The growth of global e-commerce and ever-shortening delivery promises have transformed large distribution centers into highly dynamic cyber-physical systems deploying hundreds of autonomous mobile robots (AMRs) and human pickers. In such environments, a core bottleneck is **real-time task allocation**: deciding which agent should execute which task (picking, moving, charging, etc.) to maximize throughput and avoid congestion. Formally, this problem couples: (i) stochastic arrival processes of tasks, (ii) spatially constrained path-planning domains, and (iii) resource conflicts (e.g., narrow aisles and shared charging stations).

1.2 Limitations of Existing Approaches

Traditional rule-based schedulers or mixed-integer programming approaches make strong assumptions about complete knowledge and typically re-optimize only at coarse time scales. These methods struggle when tasks arrive continuously and system states change faster than the optimizer can solve.

More recent **Multi-Agent Reinforcement Learning (MARL)** eliminates hand-crafted heuristics but suffers from the curse of dimensionality: flat MARL must explore a joint action space that grows exponentially with the number of agents and tasks.

Hierarchical MARL (HMARL) mitigates this by introducing a manager-worker structure; however, most managers rely on categorical policies that implicitly assume **Independence of Irrelevant Alternatives (IIA)**. When a high-priority task appears in one region, this assumption causes the manager to re-weight **all** alternatives—regardless of relevance—thereby increasing the risk of congestion and resource idleness elsewhere.

1.3 Core Idea

To relax IIA while preserving hierarchical decomposition, we consider equipping the high-level manager with a **Nested Logit (NL)** choice mechanism. The NL structure first selects a **nest**—e.g., all picking tasks in Region A or all charging tasks—then selects a specific task within that nest. This two-stage view preserves associations among correlated alternatives within nests and treats tasks in different nests as essentially independent, matching the spatial and functional groupings observed in real warehouses.

Designing and learning such an NL-HMARL architecture raises several open questions:

1. How to define nests from raw warehouse states?
2. How to integrate NL choice probabilities into RL updates?
3. How to keep the overall decision loop fast enough for real-time control?

1.4 Paper Scope

The remainder formalizes NL-HMARL (Section 3), describes learning (Section 3.3), and presents empirical evaluation across 6 configurations (Section 5), validating NL-HMARL's advantages over Softmax-HMARL.

2 Background and Related Work

Warehouse order picking has become a showcase domain for multi-agent decision-making: dozens of AMRs and human pickers must cooperate in narrow aisles while new orders continuously arrive. The key challenge is **real-time task allocation**—deciding **who** should execute **which** task to maintain high throughput and avoid congestion. Researchers have proposed solutions ranging from hand-crafted path rules to learning-based schedulers.

2.1 Hand-Crafted Path Heuristics

Early warehouse research proposed geometric heuristics such as **S-Shape**, **Return**, and **Largest-Gap** strategies [1]. These methods plan deterministic picker paths—e.g., entering an aisle from one end and exiting from the other—guaranteeing complete coverage with minimal computation.

Advantages: No training required, simple implementation, intuitive for practitioners.

Disadvantages: Assumes task independence; adds extra travel distance when picks are sparse or aisles are dead-ends.

2.2 Exact and Approximate Operations Research Methods

To go beyond single-picker rules, researchers turned to exact optimization. **Traveling Salesman Problem (TSP) variants** model warehouse picking as combinatorial optimization: given a set of pick locations, find the shortest path visiting all points and returning to the origin. Since TSP searches all possible path combinations through exhaustive or branch-and-bound methods, it can theoretically find global optima. However, TSP is NP-hard, and its computational complexity grows exponentially with the number of pick points [2]. Approximate dynamic programming schemes—such as partition-optimization DP or accelerated SKU classification—balance solution quality and runtime. However, most OR models must re-solve when new orders arrive, limiting their real-time availability in large distribution centers. This limitation motivates learning-based approaches: through offline training, learned methods can achieve constant-time complexity decisions at inference.

2.3 Learning-Based Single-Agent Methods

With the advent of deep RL, **DQN-style** agents have been trained for step-by-step planning on grid abstractions [3]. Such agents respond quickly but face explosive joint action spaces when multiple robots act simultaneously. Fine-grained policies also risk myopic oscillations since they lack a global task allocation perspective.

Flat (non-hierarchical) MARL directly selects low-level actions without a manager. As Canese et al. [4] note, joint action space grows exponentially with agent count ($|A|^N$), making these methods difficult to scale. Bahrpeyma and Reichelt [5] further emphasize that exploration in joint action learning is extremely difficult in large warehouse environments.

2.4 Hierarchical MARL

Hierarchical MARL (HMARL) splits the decision stack: a **manager** assigns tasks, and **worker** policies execute actions [6].

Despite high sample efficiency, most high-level managers rely on categorical softmax selection, which embeds the **Independence of Irrelevant Alternatives (IIA)** assumption. IIA means that the relative choice probability between any two options is unaffected by other options. In practice, when a highly similar new task appears, softmax disproportionately reduces probabilities of related tasks, even though the new task's addition should not affect original tasks' priorities relative to other unrelated tasks.

Example: With tasks A, B in Region 1 (utilities 2.0, 1.8) and C in Region 2 (utility 1.5), adding a new task D in Region 1 causes Softmax to reduce $P(C)$ from 25% to 18%—even though C is unrelated. Under Nested-Logit, A, B, D compete within the same nest while C's relative priority remains stable.

2.5 Modeling Correlated Tasks

The operations research literature uses **Nested Logit (NL)** structures to model correlated choices [7]. We choose NL over Mixed Logit or Probit because it maintains closed-form probabilities (enabling end-to-end training), naturally matches warehouse

spatial-functional groupings, and is computationally efficient. To our knowledge, NL with learnable nest dissimilarity parameters has not been integrated with online MARL managers.

2.6 Research Gap

The above review reveals open problems: real-time scalability under dense task arrivals, explicit modeling of task correlations, and robust high-level decisions beyond black-box softmax. We propose embedding an NL layer into HMARL, yielding a two-stage **select-nest→select-task** policy.

3 Proposed Method

3.1 Problem Formulation

We consider a large robotic warehouse with a set of AMRs and an online stream of tasks (picking, replenishment, charging, inspection). Let $t \in \{0, 1, \dots\}$ index decision epochs. The warehouse state is represented as s_t, aggregating:

- (i) Robot kinematics and battery levels
- (ii) Current task pool $\mathcal{T}_t = \{\tau_t^1, \dots, \tau_t^A\}$ with spatial locations and priorities
- (iii) Layout and resource states (aisle occupancy, charging station queues)

We denote $A = |\mathcal{T}_t|$ as the number of available tasks and partition tasks into a set of nests $\mathcal{G}_t = \{\mathcal{N}_t^1, \dots, \mathcal{N}_t^G\}$, each capturing a correlated subset (e.g., all picking tasks in a region, all charging tasks). Let $G = |\mathcal{G}_t|$ be the number of nests.

We adopt a hierarchical control scheme: a high-level manager selects a task (or idles) for each robot, and low-level worker policies execute motion actions. Let the manager's decision at time t be selecting nest $m_t \in \{1, \dots, G\}$, then selecting task $i_t \in \mathcal{N}_t^{m_t}$. Each robot k then follows worker policy $\pi_\omega(\cdot|o_t^k, i_t)$, mapping local observation o_t^k and assigned task to low-level actions. The environment returns scalar reward r_t, balancing throughput, latency penalties, path length, energy usage, collisions, and deadlocks.

Our objective is to maximize expected discounted return:

$$J(\theta, \phi, \lambda, \psi) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]$$

where $\gamma \in (0, 1)$, (θ, ϕ, λ) parameterize the manager policy, and ψ parameterizes the critic/baseline for variance reduction. The worker policy uses fixed heuristics and does not participate in learning.

3.2 Nested-Logit Hierarchical Architecture

To relax IIA while maintaining full differentiability, we equip the manager with a Nested-Logit (NL) two-stage policy.

Task Utility and Nest Dissimilarity For each candidate task $i \in \mathcal{T}_t$, we define a learnable utility:

$$u_i = u_\theta(s_t, i) \in \mathbb{R}$$

For each nest m, define a learnable dissimilarity parameter $\eta_m \in [\eta_{\min}, 1]$. We parameterize $\eta_m = \sigma(\lambda_m)$ via sigmoid to obtain values in $(0, 1)$, then clip to $[\eta_{\min}, 1]$ (where $\eta_{\min} \approx 0.1$) for numerical stability, learning $\lambda_m \in \mathbb{R}$. Define the nest inclusive value:

$$I_m = \log \sum_{j \in \mathcal{N}_t^m} \exp\left(\frac{u_j}{\eta_m}\right)$$

Two-Stage Manager Policy We introduce nest-level scores $b_m = b_\phi(s_t, m)$. The NL manager first samples a nest, then samples a task within it, with probabilities:

$$\pi_{\text{nest}}(m|s_t) = \frac{\exp(b_m + \eta_m I_m)}{\sum_n \exp(b_n + \eta_n I_n)}$$

$$\pi_{\text{task}}(i|s_t, m) = \frac{\exp(u_i/\eta_m)}{\sum_{j \in \mathcal{N}_t^m} \exp(u_j/\eta_m)}$$

In our design, each task belongs to exactly one nest (uniquely determined by its spatial location and task type). Therefore, the joint manager probability of selecting task i is:

$$\pi_{\text{M}}(i|s_t) = \pi_{\text{nest}}(m_i|s_t) \cdot \pi_{\text{task}}(i|s_t, m_i)$$

where m_i denotes the unique nest containing task i.

This structure captures intra-nest correlations through η_m while keeping tasks in different nests essentially independent.

Worker Layer Given assigned task i_t, the worker policy $\pi_\omega(\cdot|o_t^k, i_t)$ produces motion-level actions until termination (success, failure, or abort), at which point control returns to the manager for reassignment. To simplify training and accelerate convergence, the worker policy uses predefined heuristic navigation (A*-based pathfinding) rather than end-to-end learning.

3.3 Policy Representation and Learning

We train the manager policy using a simplified Advantage Actor-Critic (A2C) approach. Let $\hat{A}_t = R_t - V_\psi(s_t)$ be the advantage estimate, where $R_t = r_t + \gamma V_\psi(s_{t+1})$ is the 1-step TD return. The manager's log probability decomposes as:

$$\log \pi_{\text{M}}(i_t|s_t) = \log \pi_{\text{nest}}(m_t|s_t) + \log \pi_{\text{task}}(i_t|s_t, m_t)$$

The manager loss is:

$$\begin{aligned}\mathcal{L}_{\text{M}}(\theta, \phi, \lambda) = &-\mathbb{E}[\hat{A}_t \log \pi_{\text{M}}(i_t|s_t)] \\ &- \beta_{\text{H}} H[\pi_{\text{nest}}(\cdot|s_t)] \\ &- \beta_{\text{H}} \sum_m H[\pi_{\text{task}}(\cdot|s_t, m)]\end{aligned}$$

where β_{H} is the entropy weight. The critic minimizes $\mathcal{L}_{\mathrm{V}}(\psi) = \mathbb{E}[(R_t - V_\psi(s_t))^2]$.

Manager parameters $\{\theta, \phi, \lambda, \psi\}$ are updated via stochastic gradient descent/ascent. The NL components are fully differentiable; gradients flow through I_m, η_m, and log probabilities.

3.4 Properties and Theoretical Analysis

End-to-End Differentiability and Correlation Modeling The NL decomposition keeps the entire pipeline differentiable while allowing $\eta_m \in [\eta_{\min}, 1]$ to capture intra-nest correlations. When $\eta_m \to 1$, the model approaches softmax over all tasks; smaller η_m increases intra-nest substitutability and reduces cross-nest interference.

Real-Time Complexity Recalling the definitions of A (task count) and G (nest count) from Section 3.1: one decision requires computing utilities u_i in $O(A)$; computing I_m via a single pass grouping tasks by nest in $O(A)$; forming nest logits in $O(G)$. Thus overall inference complexity is $O(A+G)$. Since nest count G is typically much smaller than task count A, practical complexity is approximately $O(A)$. With modest batching and caching of features shared by tasks in the same region, actual latency remains compatible with millisecond-scale scheduling.

Robustness and Stability By re-weighting only alternatives in the selected nest at the second stage, the manager becomes less sensitive to irrelevant high-utility outliers elsewhere in the warehouse, improving stability under bursty arrivals. Entropy over nest selection and intra-nest selection prevents premature collapse to a single region and encourages exploration during early training.

4 Experimental Setup

4.1 Simulation Environment

We use a discrete-event warehouse simulator capturing order arrivals, robot kinematics, aisle congestion, charging constraints, and workstation service times. Environment scale and configuration are adjusted per experiment: 12×12 scale uses 20-35 pickers, 24×24 scale uses 40-70 pickers, equipped with corresponding numbers of picking workstations (3-8). Orders arrive according to a non-homogeneous Poisson process with hourly variation (peak/off-peak multipliers). Each order is decomposed into picking tasks, with item locations sampled from a heatmap fitted to typical turnover profiles. Task nests are formed online by spatial region and urgency level (4 functional regions × 2 urgency levels = 8 nests).

Robots follow simple differential-drive dynamics with maximum speed 1.2 m/s, relying on the worker layer for local collision avoidance. Charging from 20% to 80% battery takes 15 minutes. The manager replans every 2 seconds or upon early task termination. Each episode lasts 1 simulated hour; we report averages over 32 seeds.

Table 1 presents the complete experimental configuration, including core parameter differences across three difficulty levels and environment settings at different scales.

Table 1: Experimental Configuration Parameters

Category	Parameter	Config1-Easy	Config2-Medium	Config3-Hard
Order Parameters	Urgent order ratio (urgent_order_prob)	0.40	0.30	0.25
	Items per order (max_items)	1	2	3
	Burst probability (bursty_prob)	0.35	0.50	0.65
	Burst size (burst_size)	[12, 28]	[18, 40]	[25, 60]
	Base order rate (base_rate)	1,200	1,500	1,800
Environment Constraints	Zone capacity (zone_capacity)	[3,3,4,3]	[2,2,3,2]	[2,2,2,2]
	Overcapacity penalty (overcapacity_penalty)	-2.0	-5.0	-8.0
	Congestion multiplier (congestion_mult)	0.7	0.5	0.4
12×12	Number of pickers	20	30	35
	Number of workstations	5	4	3
	Number of forklifts	4	4	4
24×24	Number of pickers	40	60	70
	Number of workstations	8	8	6
	Number of forklifts	8	12	14

4.2 Evaluation Metrics

We evaluate policies using standard throughput and responsiveness metrics:

- Order throughput (orders/hour) and task completion rate (%)
- Average task waiting time and 95% tail latency (seconds)
- Congestion time (seconds within proximity threshold)
- Episode return (undiscounted cumulative reward) and safety violations (near-collisions/hour)

We also report per-region balance (coefficient of variation of queue lengths) to quantify spatial load smoothing.

4.3 Baseline Methods

We compare NL-HMARL against:

- **Rule-based heuristics**: S-Shape and Return path planning, using predefined path patterns for task allocation
- **Greedy nearest-distance method (Greedy-Nearest)**: Greedy allocation based on minimum Manhattan distance, selecting the nearest pending task for each idle picker. Note: abbreviated as "Optimal" in tables for brevity, though this method is not globally optimal
- **Softmax-based hierarchical MARL**: Same architecture as NL-HMARL, but with standard categorical softmax instead of Nested-Logit structure

Learning baselines (Softmax-HMARL) use the same training steps (10,000) and similar hyperparameter settings. Rule-based methods require no training.

4.4 Implementation Details

The manager uses a two-layer MLP (hidden sizes 256/128) for utility and nest scorers; $\eta_m = \sigma(\lambda_m)$ with initial $\lambda_m = 0$. Workers use predefined heuristic navigation (A*-based pathfinding), no learning required.

We train the manager with Adam (learning rate $= 1 \times 10^{-3}$), entropy weight $\beta_H = 0.01$, discount $\gamma = 0.99$. Parameters update every 8 manager decision steps. Training uses 10,000 steps (approximately 5-10 complete episodes depending on environment scale). Nests are recomputed at each manager decision based on task spatial region and urgency (4 functional regions $\times$ 2 urgency levels = 8 nests). This nest structure is chosen because: (1) spatial region partitioning reflects warehouse physical layout, with tasks in the same region having natural similarity in path planning; (2) urgency partitioning captures the main dimension of task value differences; (3) 8 nests balance expressiveness and computational efficiency—too few nests cannot adequately capture task correlations, too many increase learning difficulty.

Pilot experiments ran locally on MacBook Pro (M1 Pro) with PyTorch MPS enabled; full-scale training ran on Google Colab A100 GPUs. The simulator is CPU-bound; wall-clock time scales primarily with CPU throughput.

5 Experimental Results and Analysis

We conducted comprehensive experimental evaluation across 6 configurations, covering 3 difficulty levels (Config1-Easy, Config2-Medium, Config3-Hard) and 2 environment scales (12×12, 24×24). For each configuration, we compared 5 methods: NL-HMARL, Softmax-HMARL, Optimal (greedy minimum-distance optimal path), Return path planning, and S-Shape path planning. Evaluation metrics include cumulative reward (Raw Value), completed tasks (Tasks), and average value per task.

This section first presents overall performance comparison, then provides in-depth analysis of environment complexity and scale effects on NL-HMARL performance.

5.1 Overall Performance Comparison

Table 2 shows complete performance data for all methods across configurations. Among 6 configurations, NL-HMARL outperforms Softmax-HMARL in 5, achieving an overall win rate of 83.3%, validating the effectiveness of Nested-Logit structure in hierarchical task allocation. Notably, while NL-HMARL trails rule-based methods (Optimal, Return, S-Shape) in all configurations, this does not affect our core research objective—validating the advantages of Nested-Logit structure over standard Softmax in multi-agent hierarchical decision-making. Rule-based methods' lead is mainly attributed to their optimization design for specific scenarios, while learning methods pursue stronger generality and adaptability.

5.2 Environment Complexity Analysis

The three difficulty configurations are designed to create varying degrees of IIA problem scenarios. The `urgent_order_prob` parameter controls the proportion of high-value urgent tasks: Config1-Easy is set to 0.40 (40% high-value tasks, clear priority differentiation), Config2-Medium to 0.30, and Config3-Hard to 0.25 (only 25% high-value tasks, 75% regular tasks with similar utilities).

Core Condition for NL Advantage: When **task utility distribution tends toward uniformity** (i.e., lacking clear high/low priority differentiation), IIA problems become

Table 2: Performance Comparison of All Methods Across Configurations

Config	Method	12×12 Reward	12×12 Tasks	24×24 Reward	24×24 Tasks
Config1-Easy	Return	35,939	172	41,539	257
	S-Shape	34,346	183	36,350	212
	Optimal	18,809	228	44,949	441
	Softmax-HMARL	10,555	141	12,706	199
	NL-HMARL	9,351	162	13,790	206
Config2-Medium	Optimal	56,207	405	51,483	557
	S-Shape	42,864	270	59,991	328
	Return	33,060	255	58,321	350
	NL-HMARL	16,197	190	25,959	300
	Softmax-HMARL	14,901	234	19,339	288
Config3-Hard	Optimal	63,129	464	65,753	633
	S-Shape	51,886	335	70,072	412
	Return	45,039	312	65,246	404
	NL-HMARL	20,118	235	30,144	340
	Softmax-HMARL	18,932	267	19,758	235

more severe. In such scenarios, Softmax unreasonably disperses selection probabilities among tasks with similar utilities—adding a new similar task simultaneously reduces probabilities of **all** other tasks (including unrelated ones). The NL structure groups spatially adjacent or functionally related tasks into the same nest, making intra-nest tasks compete with each other without affecting inter-nest relative priorities, thereby mitigating IIA.

Our experiments reveal an important finding: NL-HMARL's performance advantage over Softmax-HMARL is positively correlated with environment complexity and scale. Table 3 summarizes NL-HMARL's performance advantage over Softmax-HMARL.

Table 3: NL-HMARL Performance Advantage over Softmax-HMARL

Difficulty Level	12×12 Adv.	24×24 Adv.	Scale-up Gain	Win Rate
Config1-Easy	−11.4%	+8.5%	+19.9pp	1/2
Config2-Medium	+8.7%	+34.2%	+25.5pp	2/2
Config3-Hard	+6.3%	+52.6%	+46.3pp	2/2
Average/Total	**+1.2%**	**+31.8%**	**+30.6pp**	**5/6**

Config1-Easy Environment Analysis In the simplest configuration (single-item orders, low congestion, high urgent task ratio), Softmax-HMARL wins by 11.4% at 12×12 scale. This is the only case where NL-HMARL underperforms Softmax. Analysis: single-item orders mean no need for complex multi-step path planning; low congestion environments require less picker coordination; high urgent task ratio (urgent_order_prob = 0.40) makes simple greedy strategies (prioritizing high-value tasks) very effective; in this scenario, IIA is not prominent because task substitutability is weak.

However, notably at 24×24 scale, NL-HMARL successfully overtakes by 8.5%, indicating that even in simple environments, scale-up can trigger NL structure advantages.

Config2-Medium Environment Analysis In medium complexity environments, NL-HMARL begins showing clear advantages, leading by 8.7% and 34.2% at 12×12 and 24×24 scales respectively. Key features: multi-item orders (max_items = 2) increase path planning complexity; medium congestion requires some picker coordination; reduced urgent task ratio (urgent_order_prob = 0.30) causes simple greedy strategies to fail; Nested structure allows the model to first make coarse-grained decisions at nest level (which region), then refine at task level—this hierarchical decision-making better suits medium-complexity coordination problems.

Config3-Hard Environment Analysis In the most complex environment, NL-HMARL's advantages fully manifest, leading by 6.3% and 52.6% at 12×12 and 24×24 scales respectively. Best performance appears in Config3-Hard 24×24: NL-HMARL achieves 30,144 cumulative reward while Softmax only achieves 19,758, leading by 52.6%; task completion gap: NL-HMARL completes 340 tasks, Softmax only 235, 44.7% more.

Environment features: multi-item orders (max_items = 3); high congestion (tight zone_capacity configuration); low urgent task ratio (urgent_order_prob = 0.25), cannot simply "grab big, release small"; high burst probability (bursty_prob = 0.65), more irregular order arrivals.

In this highly complex environment, IIA becomes a key bottleneck. When multiple tasks have similar values, Softmax's IIA assumption leads to suboptimal decisions: even adding or removing an unrelated low-value task can significantly change selection probabilities among high-value tasks. The NL structure, through nest hierarchy, first identifies groups of similar tasks (e.g., tasks in the same region), then selects within groups, effectively mitigating this problem.

5.3 Scale-Up Effects

As shown in Table 3, from 12×12 to 24×24 scale, NL-HMARL's average advantage increases dramatically from +1.2% to +31.8%, a gain of 30.6 percentage points. More importantly, at 24×24 scale, NL-HMARL achieves 100% win rate (3/3), while only 66.7% (2/3) at 12×12.

Why Does Scale-Up Enhance NL Advantage? Based on experimental observations and theoretical analysis, we propose the following possible explanations (Note: these are theoretical conjectures not yet verified through dedicated ablation experiments):

1. **Exponential growth in coordination complexity**: From 12×12 (20-35 pickers) to 24×24 (40-70 pickers), picker count increases about 2-3×, but potential task allocation combinations grow exponentially. In high-density scenarios, dozens of pickers may compete for tasks in the same region simultaneously, making IIA more prominent.
2. **Expressiveness advantage of hierarchical decisions**: Larger environments mean more tasks and more complex state spaces. NL's two-layer decision structure (nest selection → task selection) can express complex decision strategies more effectively than Softmax's single-layer decision.
3. **Value of regional partitioning**: In 24×24 environments, spatial distribution of 4 functional regions (storage, packing, charging, aisles) becomes more pronounced.

NL naturally groups tasks by region into nests, and this structured representation becomes more valuable at larger scales.

4. **Training convergence efficiency**: Although both methods use the same training steps (10,000), in large-scale environments, NL's hierarchical structure may provide better sample efficiency.

Theoretical Complexity and Inference Efficiency Comparison Recalling the theoretical analysis in Section 3.4, time complexities are: NL-HMARL and Softmax-HMARL have inference complexity $O(A+G)$ (A = task count, G = nest count), approximating $O(1)$ due to fixed neural network structure; S-Shape/Return methods need to traverse tasks for each picker for allocation and path sorting, complexity $O(P \cdot A)$ (P = picker count); Optimal method based on global distance matrix search, complexity $O(P \cdot A \cdot W \cdot H)$ ($W \times H$ = grid size).

Table 4 shows actual inference time benchmark results for each method at different scales.

Table 4: Inference Efficiency Comparison of Methods

Method	**Time Complexity**	**12×12 (ms)**	**24×24 (ms)**	**Scale Growth**
NL-HMARL	$O(A+G) \approx O(1)$	0.057	0.057	≈1×
Softmax-HMARL	$O(A) \approx O(1)$	0.060	0.060	≈1×
S-Shape/Return	$O(P \cdot A)$	0.058	0.19	≈3.3×
Optimal	$O(P \cdot A \cdot W \cdot H)$	1.98	7.86	≈4×

Key Finding: Learning methods (NL-HMARL, Softmax) have inference times nearly invariant to environment scale, maintaining ∼0.06ms/decision from 12×12 to 24×24. In contrast:

- S-Shape/Return inference time increases **3.3**× at 24×24 (from 0.058ms to 0.19ms)
- Optimal is already **138**× slower than learning methods at 24×24 (7.86ms vs 0.057ms)

This reveals the key advantage of learning methods in large-scale real-time systems: **one-time training cost in exchange for constant inference time**. For warehouse scheduling systems requiring millisecond-level responses, this property has significant engineering implications.

Large-Scale E-commerce Warehouse Outlook Real large e-commerce warehouses far exceed our experimental settings. Taking automated warehouses of leading e-commerce companies like Amazon and JD.com as examples, typical configurations are 100×100 grids or larger (approximately 10,000 square meters of working area), equipped with 500-1000 pickers (human or AGV), processing tens of thousands of orders per hour. Based on our time complexity analysis, we extrapolate inference performance at 100×100 scale:

- **NL-HMARL/Softmax-HMARL**: Due to fixed neural network structure, inference time remains ∼0.06ms/decision, **independent of scale**.
- **S-Shape/Return**: Time complexity $O(P \cdot A)$. Assuming 100×100 environment has 500 pickers and 1000 pending tasks (about 8-10× that of 24×24), inference time is expected to grow to **1-2ms/decision**, **20-30**× slower than learning methods.

- **Optimal**: Time complexity $O(P \cdot A \cdot W \cdot H)$. At 100×100 scale, grid area is 17× that of 24×24, combined with picker and task count growth, inference time is expected to reach **second-level**, **unable to meet real-time decision requirements**.

This outlook indicates: in real large-scale e-commerce warehouse scenarios, learning methods' computational efficiency advantage will expand from the several-fold gap observed in experiments to **tens of times or more**. NL-HMARL's constant inference time property makes it particularly suitable for large-scale real-time scheduling systems requiring millisecond-level responses.

6 Conclusion

This paper proposes the Nested-Logit Hierarchical Multi-Agent Reinforcement Learning (NL-HMARL) framework for real-time task allocation in robotic warehouses. Through systematic experiments across 3 difficulty levels and 2 environment scales, we draw the following core conclusions:

(1) Learning methods have computational efficiency advantages: NL-HMARL inference time is constant (∼0.06ms/decision), independent of environment scale; while the Optimal method is already 138× slower than learning methods at 24×24 scale. In 100×100-level large e-commerce warehouses, this advantage will expand to tens of times or more.

(2) NL structure outperforms Softmax in complex scenarios: When task utility distribution is uniform (severe IIA problem), NL advantages emerge. In Config3-Hard 24×24 configuration, NL-HMARL achieves 52.6% higher cumulative reward and 44.7% more task completions than Softmax. When environment scale increases, NL win rate rises from 66.7% to 100%.

Limitations: NL shows no advantage in simple environments; training steps (10,000) are relatively few; performance gap remains compared to tuned rule-based methods.

Future Work: Increase training steps, optimize network architecture (attention mechanisms, graph neural networks), explore transfer learning, validate larger scales (100×100), and automatic nest structure learning.

References

[1] H. D. Ratliff and A. S. Rosenthal, "Order-picking in a rectangular warehouse: A solvable case of the traveling salesman problem," *Operations Research*, vol. 31, no. 3, pp. 507–521, 1983.

[2] R. De Koster and E. S. Van der Poort, "Routing order pickers in a warehouse: A comparison between optimal and heuristic solutions," *IIE Transactions*, vol. 30, pp. 469–480, 1998.

[3] M. S. Alam, M. U. Khan, and A. Gunes, "Reinforcement learning-based multi-robot path planning and congestion management in warehouse order picking," in *Proc. 26th International Multi-Topic Conference (INMIC)*, 2024, pp. 1–6.

[4] L. Canese, G. C. Cardarilli, L. Di Nunzio, R. Fazzolari, D. Giardino, M. Re, and S. Spanò, "Multi-agent reinforcement learning: A review of challenges and applications," *Applied Sciences*, vol. 11, no. 11, p. 4948, 2021.

[5] F. Bahrpeyma and D. Reichelt, "A review of the applications of multi-agent reinforcement learning in smart factories," *Frontiers in Robotics and AI*, vol. 9, p. 1027340, 2022.

[6] A. Krnjaic, R. D. Steleac, J. D. Thomas, G. Papoudakis, L. Schäfer, A. W. K. To, K.-H. Lao, M. Cubuktepe, M. Haley, P. Børsting, and S. V. Albrecht, "Scalable multi-agent reinforcement learning for warehouse logistics with robotic and human co-workers," in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 677–684.

[7] K. E. Train, *Discrete Choice Methods with Simulation*, 2nd ed. Cambridge University Press, 2009.